

ABTP Sequential Codex Execution Plan

AI Blockchain Trading Platform staged prompt book for minimum-risk automated crypto trading development

Version 1.0 | Prepared from ABTP source documents | 28 July 2026

Purpose

This document converts the ABTP vision, SRS, engineering package plan, system architecture, master index, and project summary into a sequential execution plan. Each stage is written as a complete Codex-executable prompt that can be copied into Codex and completed before the next stage begins.

The practical goal is to build an AI-assisted crypto trading platform that studies many Bitcoin and cryptocurrency dependency parameters, but the governing principle is minimum risk: the system must protect capital, reject unsafe trades, and prove behavior in backtesting and paper trading before any live automation.

Source Alignment

- Master Index: project vision, capital preservation, modular architecture, AI-assisted decisions, multi-exchange support, testing and documentation.
- SRS: backtesting, paper trading, live trading, configurable risk management, audit logs, dashboard/reporting, security, observability, and maintainability.
- Engineering Package Plan: 30 detailed implementation prompts, module specifications, database/API/AI/risk/deployment/testing guides.
- System Architecture: Dashboard, API Gateway, Data Collector, Database, Indicator Engine, Feature Engineering, AI Prediction, Strategy, Risk, Portfolio, Execution, Backtesting, Paper Trading, Live Trading, Monitoring, Notification.
- Project Summary: phased delivery through Foundation, Data, AI/Intelligence, Trading, Risk, Backtesting, Paper Trading, Live Trading, and Deployment.

Financial and Safety Boundary

ABTP must be engineered as a decision-support and controlled automation system, not as a guaranteed-income machine. Automated systems and AI models cannot predict future market shocks with certainty. Crypto assets can be volatile, speculative, manipulated, technically disrupted, or affected by platform failures.

External risk references for operator awareness: [CFTC AI trading bot advisory](#); [CFTC virtual currency trading risks](#); [SEC crypto asset investor alert](#).

Architecture Rules for Every Stage

- Preserve modular architecture, loose coupling, high cohesion, testability, observability, and environment-driven configuration.
- Do not place exchange API calls outside exchange adapter modules.
- Do not allow any order path to bypass the Risk Management Engine.
- Prefer spot trading first; leverage, margin, futures, and options stay disabled unless a later explicit stage enables them.
- Every decision must be explainable through stored inputs, generated signals, risk checks, and audit events.
- Use deterministic tests with fixtures; do not require real exchange credentials for unit tests.

Bitcoin and Crypto Dependency Parameter Catalogue

No bot can study literally every factor in the world, so ABTP should instead implement an extensible parameter registry. The registry lets the platform add new data sources without rewriting strategies or risk controls.

Parameter Group	Examples to Support
Price action	OHLCV candles, returns, gaps, candle body/wick ratios, support/resistance, breakouts, reversals, market structure highs/lows.
Liquidity and microstructure	Bid-ask spread, order-book depth, imbalance, recent trade flow, estimated slippage, exchange fees, minimum order rules.
Trend and momentum	Moving averages, EMA/SMA crossovers, MACD, RSI, stochastic, ADX, rate of change, multi-timeframe confirmation.
Volatility	ATR, realized volatility, Bollinger Bands, volatility regime, abnormal candle range, volatility expansion/contraction.
Volume	Volume spikes, volume moving averages, VWAP, OBV, accumulation/distribution, buy/sell taker pressure where available.
Derivatives context	Funding rates, open interest, basis, long/short ratios, liquidation clusters; collect as optional signals, not required for spot-only MVP.
On-chain context	Transaction volume, active addresses, exchange inflow/outflow, miner behavior, fees, hash-rate/protocol health where reliable providers exist.
Cross-asset and market context	BTC dominance, ETH/BTC trend, total crypto market cap, stablecoin supply/flows, correlations among selected assets.
Macro and calendar context	Interest-rate events, inflation releases, liquidity proxies, major market holidays, high-impact scheduled announcements.

News and sentiment	Verified news events, social sentiment, fear/greed proxies, scam/manipulation warnings; treat as low-trust unless source confidence is high.
Exchange and infrastructure health	API latency, outage status, websocket disconnects, stale data, rejected orders, wallet/deposit/withdrawal warnings.
Portfolio state	Current exposure, unrealized P/L, drawdown, correlated positions, cash reserve, concentration, open orders, recent losses.
Data quality	Missing candles, duplicate records, timestamp drift, outliers, stale feeds, provider disagreement, schema version.

Minimum-Risk Control Policy

- Default mode is research/backtest/paper; live trading remains locked until final stages and manual approval.
- Default instrument scope is spot crypto only. No leverage by default.
- Every trade requires a stop-loss, position-size calculation, reward-to-risk check, slippage estimate, fee estimate, and audit record.
- Suggested initial defaults: max risk per trade 0.25 percent of trading capital, daily loss halt 1 percent, weekly loss halt 3 percent, max account drawdown halt 5 percent.
- Reject trades during stale data, exchange outage, abnormal spread, excessive slippage, parameter disagreement, news shock, or risk-engine failure.
- Require paper-trading evidence before live trading: at least 30 calendar days or 100 simulated trades, positive expectancy after fees, acceptable drawdown, and stable behavior across market regimes.

Sequential Stage Map

Stage	Phase	Codex Prompt
005	Foundation	Repository Foundation and Engineering Standards
006	Foundation	Configuration, Secrets, and Environment Profiles
007	Foundation	Core Domain Models and Module Interfaces
008	Foundation	Database Schema, Migrations, and Repositories
009	Data	Exchange Adapter Contracts and Sandbox Connector
010	Data	Historical Market Data Collector
011	Data	Live Market Data Stream

012	Data	Data Quality and Normalization Engine
013	Data	Bitcoin Dependency Parameter Registry
014	Intelligence	Technical Indicator Engine
015	Intelligence	Feature Engineering Pipeline
016	Intelligence	External Context Ingestion Interfaces
017	Intelligence	AI Dataset Builder and Baseline Model
018	Intelligence	Market Regime Classifier
019	Intelligence	Prediction Service and Explainability Layer
020	Trading	Strategy Engine Framework
021	Trading	Minimum-Risk Spot Strategy V1
022	Risk	Risk Management Engine
023	Risk	Portfolio and Position Manager
024	Trading	Order Execution Engine in Paper-Safe Mode
025	Validation	Backtesting Engine
026	Validation	Backtesting Reports and Performance Metrics
027	Validation	Paper Trading Engine
028	Validation	Paper Trading Dashboard and API
029	Production	Logging, Monitoring, and Audit Trail
030	Production	Alerts and Notification Service
031	Production	Security Hardening and Credential Protection
032	Production	Supervised Live Trading Gateway
033	Production	Limited Automation and Circuit Breakers
034	Production	Final Integration, Release Checklist, and User Guide

How to Execute These Prompts

1. Run one stage at a time in Codex.
2. Do not start the next stage until the previous stage's tests and acceptance criteria pass.
3. If Codex discovers an existing codebase, it should adapt to the existing structure while preserving the architecture rules.

4. For every stage, require a completion summary that lists files changed, tests run, assumptions, and remaining risks.
5. Never enable live trading until the supervised live gateway and limited automation stages are completed and reviewed.

Codex-Executable Stage Prompts

Copy one full prompt at a time. Each prompt already includes objectives, prerequisites, files, architecture constraints, parameter focus, minimum-risk controls, tests, and acceptance criteria.

Stage 005 - Repository Foundation and Engineering Standards

Phase: Foundation

You are Codex working in the ABTP repository. Implement Stage 005: Repository Foundation and Engineering Standards.

Objective:

Create the base ABTP repository structure, coding standards, development workflow, and project documentation skeleton.

Prerequisites:

ABTP source documents available; no existing production code required.

First actions:

1. Inspect the repository structure, existing documentation, tests, and configuration.
2. Preserve existing user work and follow existing project patterns where they exist.
3. If files already exist, modify them carefully instead of replacing unrelated work.

Files to create or modify:

README.md; pyproject.toml/package config; .env.example; docs/architecture.md;
docs/codex_workflow.md; src/abtp/__init__.py; tests/README.md.

Architecture constraints:

- Preserve modular architecture, loose coupling, high cohesion, testability, observability, and environment-driven configuration.
- Do not place exchange API calls outside exchange adapter modules.
- Do not allow any order path to bypass the Risk Management Engine.
- Prefer spot trading first; leverage, margin, futures, and options stay disabled unless a later explicit stage enables them.
- Every decision must be explainable through stored inputs, generated signals, risk checks, and audit events.
- Use deterministic tests with fixtures; do not require real exchange credentials for unit tests.

Data and Bitcoin-dependency parameters:

No trading parameters yet. Define shared vocabulary for assets, exchanges, candles, signals, risks, orders, portfolios, and audit events.

Minimum-risk controls:

Live trading disabled by default; create SAFE_MODE=true configuration; document that no profit is guaranteed and only risk capital may be used.

Implementation requirements:

1. Keep the module narrowly scoped to this stage.
2. Define clear interfaces and data contracts.
3. Add documentation explaining assumptions, configuration, and operating limits.
4. Ensure future stages can extend the module without tight coupling.

Tests to add or run:

Project import test, lint/type-check command, formatting check, CI smoke test.

Acceptance criteria:

A fresh clone can install dependencies, run tests, read documentation, and confirm live execution is impossible.

Completion response:

Summarize created/modified files, tests run, remaining risks, and any assumptions. Do not claim the bot is profitable; report only verified behavior.

Stage 006 - Configuration, Secrets, and Environment Profiles

Phase: Foundation

You are Codex working in the ABTP repository. Implement Stage 006: Configuration, Secrets, and Environment Profiles.

Objective:

Implement typed configuration for research, backtest, paper, and live profiles without exposing credentials.

Prerequisites:

Stage 005 complete.

First actions:

1. Inspect the repository structure, existing documentation, tests, and configuration.
2. Preserve existing user work and follow existing project patterns where they exist.
3. If files already exist, modify them carefully instead of replacing unrelated work.

Files to create or modify:

src/abtp/config/settings.py; src/abtp/config/profiles.py;
src/abtp/config/validation.py; .env.example; docs/configuration.md; tests/config/.

Architecture constraints:

- Preserve modular architecture, loose coupling, high cohesion, testability, observability, and environment-driven configuration.
- Do not place exchange API calls outside exchange adapter modules.
- Do not allow any order path to bypass the Risk Management Engine.
- Prefer spot trading first; leverage, margin, futures, and options stay disabled unless a later explicit stage enables them.
- Every decision must be explainable through stored inputs, generated signals, risk checks, and audit events.
- Use deterministic tests with fixtures; do not require real exchange credentials for unit tests.

Data and Bitcoin-dependency parameters:

Risk limits, exchange names, asset universe, candle intervals, data providers, fee assumptions, slippage ceilings, paper/live flags.

Minimum-risk controls:

Default profile must be paper or research. Live profile requires explicit environment variable, manual confirmation flag, and separate credentials.

Implementation requirements:

1. Keep the module narrowly scoped to this stage.
2. Define clear interfaces and data contracts.
3. Add documentation explaining assumptions, configuration, and operating limits.
4. Ensure future stages can extend the module without tight coupling.

Tests to add or run:

Validation tests for missing secrets, unsafe live flags, invalid risk values, and profile precedence.

Acceptance criteria:

Unsafe configs fail fast with clear messages and no module can infer live mode implicitly.

Completion response:

Summarize created/modified files, tests run, remaining risks, and any assumptions. Do not claim the bot is profitable; report only verified behavior.

Stage 007 - Core Domain Models and Module Interfaces

Phase: Foundation

You are Codex working in the ABTP repository. Implement Stage 007: Core Domain Models and Module Interfaces.

Objective:

Define typed domain models and interfaces used across data, AI, strategy, risk, portfolio, and execution modules.

Prerequisites:

Stages 005-006 complete.

First actions:

1. Inspect the repository structure, existing documentation, tests, and configuration.
2. Preserve existing user work and follow existing project patterns where they exist.
3. If files already exist, modify them carefully instead of replacing unrelated work.

Files to create or modify:

src/abtp/domain/models.py; src/abtp/domain/enums.py; src/abtp/interfaces/*.py;
docs/module_contracts.md; tests/domain/.

Architecture constraints:

- Preserve modular architecture, loose coupling, high cohesion, testability, observability, and environment-driven configuration.
- Do not place exchange API calls outside exchange adapter modules.
- Do not allow any order path to bypass the Risk Management Engine.
- Prefer spot trading first; leverage, margin, futures, and options stay disabled unless a later explicit stage enables them.
- Every decision must be explainable through stored inputs, generated signals, risk checks, and audit events.
- Use deterministic tests with fixtures; do not require real exchange credentials for unit tests.

Data and Bitcoin-dependency parameters:

AssetPair, Candle, OrderBookSnapshot, Trade, FeatureVector, Prediction, Signal, RiskDecision, OrderIntent, PortfolioSnapshot.

Minimum-risk controls:

RiskDecision must include allow/reject, reasons, max position size, stop-loss requirement, and kill-switch status.

Implementation requirements:

1. Keep the module narrowly scoped to this stage.
2. Define clear interfaces and data contracts.
3. Add documentation explaining assumptions, configuration, and operating limits.
4. Ensure future stages can extend the module without tight coupling.

Tests to add or run:
Serialization, validation, enum, interface contract, and backward-compatible schema tests.

Acceptance criteria:
All future modules can exchange typed objects without circular imports or direct database coupling.

Completion response:
Summarize created/modified files, tests run, remaining risks, and any assumptions. Do not claim the bot is profitable; report only verified behavior.

Stage 008 - Database Schema, Migrations, and Repositories

Phase: Foundation

You are Codex working in the ABTP repository. Implement Stage 008: Database Schema, Migrations, and Repositories.

Objective:
Create persistent storage for market data, features, predictions, signals, risk decisions, orders, positions, and audit logs.

Prerequisites:
Stage 007 complete.

First actions:

1. Inspect the repository structure, existing documentation, tests, and configuration.
2. Preserve existing user work and follow existing project patterns where they exist.
3. If files already exist, modify them carefully instead of replacing unrelated work.

Files to create or modify:
migrations/; src/abtp/db/session.py; src/abtp/db/models.py; src/abtp/repositories/*.py; docs/database_design.md; tests/db/.

Architecture constraints:

- Preserve modular architecture, loose coupling, high cohesion, testability, observability, and environment-driven configuration.
- Do not place exchange API calls outside exchange adapter modules.
- Do not allow any order path to bypass the Risk Management Engine.
- Prefer spot trading first; leverage, margin, futures, and options stay disabled unless a later explicit stage enables them.
- Every decision must be explainable through stored inputs, generated signals, risk checks, and audit events.
- Use deterministic tests with fixtures; do not require real exchange credentials for unit tests.

Data and Bitcoin-dependency parameters:
Candles, order books, indicator values, feature snapshots, data quality flags, model versions, trade lifecycle, P/L, risk limits.

Minimum-risk controls:
Audit tables must be append-only for trade decisions; live credentials must never be stored in database rows.

Implementation requirements:

1. Keep the module narrowly scoped to this stage.
2. Define clear interfaces and data contracts.

3. Add documentation explaining assumptions, configuration, and operating limits.
4. Ensure future stages can extend the module without tight coupling.

Tests to add or run:

Migration up/down tests, repository CRUD tests, constraint tests, timestamp/order uniqueness tests.

Acceptance criteria:

Database can be recreated from migrations and every trading decision can be reconstructed from stored records.

Completion response:

Summarize created/modified files, tests run, remaining risks, and any assumptions. Do not claim the bot is profitable; report only verified behavior.

Stage 009 - Exchange Adapter Contracts and Sandbox Connector

Phase: Data

You are Codex working in the ABTP repository. Implement Stage 009: Exchange Adapter Contracts and Sandbox Connector.

Objective:

Create a unified exchange adapter layer and a sandbox/fake connector for tests before any real exchange integration.

Prerequisites:

Stages 006-008 complete.

First actions:

1. Inspect the repository structure, existing documentation, tests, and configuration.
2. Preserve existing user work and follow existing project patterns where they exist.
3. If files already exist, modify them carefully instead of replacing unrelated work.

Files to create or modify:

src/abtp/exchanges/base.py; src/abtp/exchanges/sandbox.py; src/abtp/exchanges/errors.py; docs/exchange_adapter.md; tests/exchanges/.

Architecture constraints:

- Preserve modular architecture, loose coupling, high cohesion, testability, observability, and environment-driven configuration.
- Do not place exchange API calls outside exchange adapter modules.
- Do not allow any order path to bypass the Risk Management Engine.
- Prefer spot trading first; leverage, margin, futures, and options stay disabled unless a later explicit stage enables them.
- Every decision must be explainable through stored inputs, generated signals, risk checks, and audit events.
- Use deterministic tests with fixtures; do not require real exchange credentials for unit tests.

Data and Bitcoin-dependency parameters:

Symbols, tick sizes, lot sizes, fees, balances, order status, rate limits, latency, websocket state.

Minimum-risk controls:

Adapters cannot execute live orders unless mode=live and risk-approved OrderIntent is supplied.

Implementation requirements:

1. Keep the module narrowly scoped to this stage.
2. Define clear interfaces and data contracts.
3. Add documentation explaining assumptions, configuration, and operating limits.
4. Ensure future stages can extend the module without tight coupling.

Tests to add or run:

Contract tests, fake order lifecycle tests, rate-limit behavior, rejected-order handling.

Acceptance criteria:

Data and execution modules can depend on adapter interfaces without knowing exchange-specific APIs.

Completion response:

Summarize created/modified files, tests run, remaining risks, and any assumptions. Do not claim the bot is profitable; report only verified behavior.

Stage 010 - Historical Market Data Collector

Phase: Data

You are Codex working in the ABTP repository. Implement Stage 010: Historical Market Data Collector.

Objective:

Collect and store historical OHLCV candles for Bitcoin and configured crypto assets.

Prerequisites:

Stage 009 complete.

First actions:

1. Inspect the repository structure, existing documentation, tests, and configuration.
2. Preserve existing user work and follow existing project patterns where they exist.
3. If files already exist, modify them carefully instead of replacing unrelated work.

Files to create or modify:

src/abtp/data/historical_collector.py; src/abtp/data/scheduler.py; src/abtp/data/symbols.py; docs/data_collection.md; tests/data/.

Architecture constraints:

- Preserve modular architecture, loose coupling, high cohesion, testability, observability, and environment-driven configuration.
- Do not place exchange API calls outside exchange adapter modules.
- Do not allow any order path to bypass the Risk Management Engine.
- Prefer spot trading first; leverage, margin, futures, and options stay disabled unless a later explicit stage enables them.
- Every decision must be explainable through stored inputs, generated signals, risk checks, and audit events.
- Use deterministic tests with fixtures; do not require real exchange credentials for unit tests.

Data and Bitcoin-dependency parameters:

OHLCV candles across 1m, 5m, 15m, 1h, 4h, 1d intervals; volume; missing candle status; provider timestamp.

Minimum-risk controls:

Collector is read-only; reject trading actions from data jobs; preserve raw provider payloads for audit where practical.

Implementation requirements:

1. Keep the module narrowly scoped to this stage.
2. Define clear interfaces and data contracts.
3. Add documentation explaining assumptions, configuration, and operating limits.
4. Ensure future stages can extend the module without tight coupling.

Tests to add or run:

Backfill pagination, duplicate prevention, gap detection, timezone normalization, retry behavior.

Acceptance criteria:

BTC historical candles can be backfilled idempotently with data-quality flags.

Completion response:

Summarize created/modified files, tests run, remaining risks, and any assumptions. Do not claim the bot is profitable; report only verified behavior.

Stage 011 - Live Market Data Stream

Phase: Data

You are Codex working in the ABTP repository. Implement Stage 011: Live Market Data Stream.

Objective:

Implement live ticker, candle, trade, and order-book ingestion through exchange adapters.

Prerequisites:

Stage 010 complete.

First actions:

1. Inspect the repository structure, existing documentation, tests, and configuration.
2. Preserve existing user work and follow existing project patterns where they exist.
3. If files already exist, modify them carefully instead of replacing unrelated work.

Files to create or modify:

src/abtp/data/live_stream.py; src/abtp/data/order_book.py; src/abtp/data/heartbeat.py;
docs/live_data.md; tests/data_live/.

Architecture constraints:

- Preserve modular architecture, loose coupling, high cohesion, testability, observability, and environment-driven configuration.
- Do not place exchange API calls outside exchange adapter modules.
- Do not allow any order path to bypass the Risk Management Engine.
- Prefer spot trading first; leverage, margin, futures, and options stay disabled unless a later explicit stage enables them.
- Every decision must be explainable through stored inputs, generated signals, risk checks, and audit events.
- Use deterministic tests with fixtures; do not require real exchange credentials for unit tests.

Data and Bitcoin-dependency parameters:

Bid/ask, spread, depth, imbalance, recent trades, candle updates, stream latency, disconnect counts.

Minimum-risk controls:

Mark data stale after configurable threshold; downstream strategies must reject stale or degraded data.

Implementation requirements:

1. Keep the module narrowly scoped to this stage.
2. Define clear interfaces and data contracts.
3. Add documentation explaining assumptions, configuration, and operating limits.
4. Ensure future stages can extend the module without tight coupling.

Tests to add or run:

Websocket reconnect simulation, stale heartbeat test, order-book snapshot/delta consistency.

Acceptance criteria:

Live feed updates normalized data objects and emits health status without placing orders.

Completion response:

Summarize created/modified files, tests run, remaining risks, and any assumptions. Do not claim the bot is profitable; report only verified behavior.

Stage 012 - Data Quality and Normalization Engine

Phase: Data

You are Codex working in the ABTP repository. Implement Stage 012: Data Quality and Normalization Engine.

Objective:

Build validation rules that detect missing, stale, duplicate, abnormal, or conflicting market data.

Prerequisites:

Stages 010-011 complete.

First actions:

1. Inspect the repository structure, existing documentation, tests, and configuration.
2. Preserve existing user work and follow existing project patterns where they exist.
3. If files already exist, modify them carefully instead of replacing unrelated work.

Files to create or modify:

src/abtp/data/quality.py; src/abtp/data/normalization.py; src/abtp/data/anomaly.py;
docs/data_quality.md; tests/data_quality/.

Architecture constraints:

- Preserve modular architecture, loose coupling, high cohesion, testability, observability, and environment-driven configuration.
- Do not place exchange API calls outside exchange adapter modules.
- Do not allow any order path to bypass the Risk Management Engine.
- Prefer spot trading first; leverage, margin, futures, and options stay disabled unless a later explicit stage enables them.
- Every decision must be explainable through stored inputs, generated signals, risk checks, and audit events.
- Use deterministic tests with fixtures; do not require real exchange credentials for unit tests.

Data and Bitcoin-dependency parameters:

Missing candles, duplicate timestamps, outlier returns, zero volume, timestamp drift, provider disagreement, abnormal spread.

Minimum-risk controls:

Risk engine must receive data-quality status and reject trades when quality is below

threshold.

Implementation requirements:

1. Keep the module narrowly scoped to this stage.
2. Define clear interfaces and data contracts.
3. Add documentation explaining assumptions, configuration, and operating limits.
4. Ensure future stages can extend the module without tight coupling.

Tests to add or run:

Fixture-based anomaly tests, malformed payload tests, degraded feed tests.

Acceptance criteria:

Every feature and signal can trace whether its source data was trusted, degraded, or rejected.

Completion response:

Summarize created/modified files, tests run, remaining risks, and any assumptions. Do not claim the bot is profitable; report only verified behavior.

Stage 013 - Bitcoin Dependency Parameter Registry

Phase: Data

You are Codex working in the ABTP repository. Implement Stage 013: Bitcoin Dependency Parameter Registry.

Objective:

Create an extensible registry of Bitcoin and crypto dependency parameters that future indicators, AI models, and strategies can consume.

Prerequisites:

Stage 012 complete.

First actions:

1. Inspect the repository structure, existing documentation, tests, and configuration.
2. Preserve existing user work and follow existing project patterns where they exist.
3. If files already exist, modify them carefully instead of replacing unrelated work.

Files to create or modify:

src/abtp/parameters/registry.py; src/abtp/parameters/catalog.py;
src/abtp/parameters/sources.py; docs/bitcoin_parameter_catalog.md; tests/parameters/.

Architecture constraints:

- Preserve modular architecture, loose coupling, high cohesion, testability, observability, and environment-driven configuration.
- Do not place exchange API calls outside exchange adapter modules.
- Do not allow any order path to bypass the Risk Management Engine.
- Prefer spot trading first; leverage, margin, futures, and options stay disabled unless a later explicit stage enables them.
- Every decision must be explainable through stored inputs, generated signals, risk checks, and audit events.
- Use deterministic tests with fixtures; do not require real exchange credentials for unit tests.

Data and Bitcoin-dependency parameters:

Register price action, liquidity, volatility, trend, volume, derivatives, on-chain, cross-asset, macro, news/sentiment, exchange health, portfolio, and data-quality parameters.

Minimum-risk controls:

Each parameter must define trust level, source, refresh interval, failure behavior, and whether it is allowed in live decisions.

Implementation requirements:

1. Keep the module narrowly scoped to this stage.
2. Define clear interfaces and data contracts.
3. Add documentation explaining assumptions, configuration, and operating limits.
4. Ensure future stages can extend the module without tight coupling.

Tests to add or run:

Registry lookup, schema validation, missing-source handling, allowed/blocked live parameter tests.

Acceptance criteria:

New parameters can be added without changing strategy or risk engine internals.

Completion response:

Summarize created/modified files, tests run, remaining risks, and any assumptions. Do not claim the bot is profitable; report only verified behavior.

Stage 014 - Technical Indicator Engine

Phase: Intelligence

You are Codex working in the ABTP repository. Implement Stage 014: Technical Indicator Engine.

Objective:

Implement reusable technical indicators over normalized market data.

Prerequisites:

Stage 013 complete.

First actions:

1. Inspect the repository structure, existing documentation, tests, and configuration.
2. Preserve existing user work and follow existing project patterns where they exist.
3. If files already exist, modify them carefully instead of replacing unrelated work.

Files to create or modify:

src/abtp/indicators/engine.py; src/abtp/indicators/trend.py; src/abtp/indicators/momentum.py; src/abtp/indicators/volatility.py; docs/indicators.md; tests/indicators/.

Architecture constraints:

- Preserve modular architecture, loose coupling, high cohesion, testability, observability, and environment-driven configuration.
- Do not place exchange API calls outside exchange adapter modules.
- Do not allow any order path to bypass the Risk Management Engine.
- Prefer spot trading first; leverage, margin, futures, and options stay disabled unless a later explicit stage enables them.
- Every decision must be explainable through stored inputs, generated signals, risk checks, and audit events.
- Use deterministic tests with fixtures; do not require real exchange credentials for unit tests.

Data and Bitcoin-dependency parameters:

SMA, EMA, RSI, MACD, ATR, Bollinger Bands, VWAP, ADX, stochastic, OBV, support/resistance helpers.

Minimum-risk controls:

Indicators emit values only; no indicator can directly create an executable order.

Implementation requirements:

1. Keep the module narrowly scoped to this stage.
2. Define clear interfaces and data contracts.
3. Add documentation explaining assumptions, configuration, and operating limits.
4. Ensure future stages can extend the module without tight coupling.

Tests to add or run:

Known-value indicator tests, warmup-window tests, NaN/missing-data tests.

Acceptance criteria:

Indicators are deterministic, versioned, and available for backtesting and live pipelines.

Completion response:

Summarize created/modified files, tests run, remaining risks, and any assumptions. Do not claim the bot is profitable; report only verified behavior.

Stage 015 - Feature Engineering Pipeline

Phase: Intelligence

You are Codex working in the ABTP repository. Implement Stage 015: Feature Engineering Pipeline.

Objective:

Transform raw data and indicators into model-ready feature vectors with versioned schemas.

Prerequisites:

Stage 014 complete.

First actions:

1. Inspect the repository structure, existing documentation, tests, and configuration.
2. Preserve existing user work and follow existing project patterns where they exist.
3. If files already exist, modify them carefully instead of replacing unrelated work.

Files to create or modify:

src/abtp/features/pipeline.py; src/abtp/features/schema.py; src/abtp/features/store.py;
docs/feature_engineering.md; tests/features/.

Architecture constraints:

- Preserve modular architecture, loose coupling, high cohesion, testability, observability, and environment-driven configuration.
- Do not place exchange API calls outside exchange adapter modules.
- Do not allow any order path to bypass the Risk Management Engine.
- Prefer spot trading first; leverage, margin, futures, and options stay disabled unless a later explicit stage enables them.
- Every decision must be explainable through stored inputs, generated signals, risk checks, and audit events.
- Use deterministic tests with fixtures; do not require real exchange credentials for unit tests.

Data and Bitcoin-dependency parameters:

Multi-timeframe returns, trend state, volatility regime, liquidity state, spread/slippage estimate, volume anomaly, portfolio context.

Minimum-risk controls:

Feature vectors must carry data-quality status and cannot be used for live signals if stale or

untrusted.

Implementation requirements:

1. Keep the module narrowly scoped to this stage.
2. Define clear interfaces and data contracts.
3. Add documentation explaining assumptions, configuration, and operating limits.
4. Ensure future stages can extend the module without tight coupling.

Tests to add or run:

Feature schema tests, leakage prevention tests, reproducibility tests, backfill/live parity tests.

Acceptance criteria:

The same feature definition works in historical backtests and live decision cycles.

Completion response:

Summarize created/modified files, tests run, remaining risks, and any assumptions. Do not claim the bot is profitable; report only verified behavior.

Stage 016 - External Context Ingestion Interfaces

Phase: Intelligence

You are Codex working in the ABTP repository. Implement Stage 016: External Context Ingestion Interfaces.

Objective:

Add optional ingestion interfaces for on-chain, macro, derivatives, and sentiment providers without hard-coding vendors.

Prerequisites:

Stage 015 complete.

First actions:

1. Inspect the repository structure, existing documentation, tests, and configuration.
2. Preserve existing user work and follow existing project patterns where they exist.
3. If files already exist, modify them carefully instead of replacing unrelated work.

Files to create or modify:

src/abtp/context/base.py; src/abtp/context/onchain.py; src/abtp/context/macro.py;
src/abtp/context/sentiment.py; docs/external_context.md; tests/context/.

Architecture constraints:

- Preserve modular architecture, loose coupling, high cohesion, testability, observability, and environment-driven configuration.
- Do not place exchange API calls outside exchange adapter modules.
- Do not allow any order path to bypass the Risk Management Engine.
- Prefer spot trading first; leverage, margin, futures, and options stay disabled unless a later explicit stage enables them.
- Every decision must be explainable through stored inputs, generated signals, risk checks, and audit events.
- Use deterministic tests with fixtures; do not require real exchange credentials for unit tests.

Data and Bitcoin-dependency parameters:

Funding rate, open interest, liquidations, active addresses, exchange flows, major calendar events, verified news/sentiment scores.

Minimum-risk controls:

External context is optional and low-trust by default; provider failure must degrade signals, not crash the platform.

Implementation requirements:

1. Keep the module narrowly scoped to this stage.
2. Define clear interfaces and data contracts.
3. Add documentation explaining assumptions, configuration, and operating limits.
4. Ensure future stages can extend the module without tight coupling.

Tests to add or run:

Provider contract tests, missing-provider tests, stale-context tests, confidence scoring tests.

Acceptance criteria:

Strategies can use external context when available while remaining functional without it.

Completion response:

Summarize created/modified files, tests run, remaining risks, and any assumptions. Do not claim the bot is profitable; report only verified behavior.

Stage 017 - AI Dataset Builder and Baseline Model

Phase: Intelligence

You are Codex working in the ABTP repository. Implement Stage 017: AI Dataset Builder and Baseline Model.

Objective:

Build datasets and a conservative baseline prediction model for research and backtesting.

Prerequisites:

Stage 016 complete.

First actions:

1. Inspect the repository structure, existing documentation, tests, and configuration.
2. Preserve existing user work and follow existing project patterns where they exist.
3. If files already exist, modify them carefully instead of replacing unrelated work.

Files to create or modify:

src/abtp/ai/datasets.py; src/abtp/ai/baseline.py; src/abtp/ai/metrics.py; docs/ai_design.md; tests/ai/.

Architecture constraints:

- Preserve modular architecture, loose coupling, high cohesion, testability, observability, and environment-driven configuration.
- Do not place exchange API calls outside exchange adapter modules.
- Do not allow any order path to bypass the Risk Management Engine.
- Prefer spot trading first; leverage, margin, futures, and options stay disabled unless a later explicit stage enables them.
- Every decision must be explainable through stored inputs, generated signals, risk checks, and audit events.
- Use deterministic tests with fixtures; do not require real exchange credentials for unit tests.

Data and Bitcoin-dependency parameters:

Feature vectors, labels for future return/volatility, market regime, fees/slippage assumptions, train/validation/test splits.

Minimum-risk controls:

Prevent look-ahead bias and data leakage; AI output is probability/confidence only, never a direct order.

Implementation requirements:

1. Keep the module narrowly scoped to this stage.
2. Define clear interfaces and data contracts.
3. Add documentation explaining assumptions, configuration, and operating limits.
4. Ensure future stages can extend the module without tight coupling.

Tests to add or run:

Split integrity tests, label leakage tests, deterministic training smoke test, metric calculation tests.

Acceptance criteria:

A baseline model can train on historical data and produce auditable predictions with documented limitations.

Completion response:

Summarize created/modified files, tests run, remaining risks, and any assumptions. Do not claim the bot is profitable; report only verified behavior.

Stage 018 - Market Regime Classifier

Phase: Intelligence

You are Codex working in the ABTP repository. Implement Stage 018: Market Regime Classifier.

Objective:

Classify market conditions so strategies and risk controls can behave differently in trend, range, high-volatility, and shock regimes.

Prerequisites:

Stage 017 complete.

First actions:

1. Inspect the repository structure, existing documentation, tests, and configuration.
2. Preserve existing user work and follow existing project patterns where they exist.
3. If files already exist, modify them carefully instead of replacing unrelated work.

Files to create or modify:

src/abtp/ai/regime.py; src/abtp/ai/regime_rules.py; docs/market_regimes.md; tests/ai_regime/.

Architecture constraints:

- Preserve modular architecture, loose coupling, high cohesion, testability, observability, and environment-driven configuration.
- Do not place exchange API calls outside exchange adapter modules.
- Do not allow any order path to bypass the Risk Management Engine.
- Prefer spot trading first; leverage, margin, futures, and options stay disabled unless a later explicit stage enables them.
- Every decision must be explainable through stored inputs, generated signals, risk checks, and audit events.
- Use deterministic tests with fixtures; do not require real exchange credentials for unit tests.

Data and Bitcoin-dependency parameters:

Trend strength, volatility percentile, volume anomaly, drawdown, correlation, spread,

liquidity, sudden news/context flags.

Minimum-risk controls:

High-volatility or shock regime should automatically tighten size or block new entries.

Implementation requirements:

1. Keep the module narrowly scoped to this stage.
2. Define clear interfaces and data contracts.
3. Add documentation explaining assumptions, configuration, and operating limits.
4. Ensure future stages can extend the module without tight coupling.

Tests to add or run:

Rule/classifier fixtures, boundary tests, regime transition tests.

Acceptance criteria:

Each decision cycle includes a regime label, confidence, and risk adjustment suggestion.

Completion response:

Summarize created/modified files, tests run, remaining risks, and any assumptions. Do not claim the bot is profitable; report only verified behavior.

Stage 019 - Prediction Service and Explainability Layer

Phase: Intelligence

You are Codex working in the ABTP repository. Implement Stage 019: Prediction Service and Explainability Layer.

Objective:

Expose AI predictions through a stable service that records model version, inputs, confidence, and explanation.

Prerequisites:

Stage 018 complete.

First actions:

1. Inspect the repository structure, existing documentation, tests, and configuration.
2. Preserve existing user work and follow existing project patterns where they exist.
3. If files already exist, modify them carefully instead of replacing unrelated work.

Files to create or modify:

src/abtp/ai/prediction_service.py; src/abtp/ai/explainability.py; src/abtp/api/predictions.py; docs/prediction_service.md; tests/prediction_service/.

Architecture constraints:

- Preserve modular architecture, loose coupling, high cohesion, testability, observability, and environment-driven configuration.
- Do not place exchange API calls outside exchange adapter modules.
- Do not allow any order path to bypass the Risk Management Engine.
- Prefer spot trading first; leverage, margin, futures, and options stay disabled unless a later explicit stage enables them.
- Every decision must be explainable through stored inputs, generated signals, risk checks, and audit events.
- Use deterministic tests with fixtures; do not require real exchange credentials for unit tests.

Data and Bitcoin-dependency parameters:

Probability of upward/downward move, expected volatility, confidence, feature importance,

regime, data-quality score.

Minimum-risk controls:

Predictions below confidence threshold or with degraded inputs must be marked non-actionable.

Implementation requirements:

1. Keep the module narrowly scoped to this stage.
2. Define clear interfaces and data contracts.
3. Add documentation explaining assumptions, configuration, and operating limits.
4. Ensure future stages can extend the module without tight coupling.

Tests to add or run:

Service contract tests, versioning tests, low-confidence rejection tests, audit persistence tests.

Acceptance criteria:

Strategy modules can request predictions without knowing model internals.

Completion response:

Summarize created/modified files, tests run, remaining risks, and any assumptions. Do not claim the bot is profitable; report only verified behavior.

Stage 020 - Strategy Engine Framework

Phase: Trading

You are Codex working in the ABTP repository. Implement Stage 020: Strategy Engine Framework.

Objective:

Create a plug-in strategy framework that turns indicators and predictions into non-executable trade signals.

Prerequisites:

Stage 019 complete.

First actions:

1. Inspect the repository structure, existing documentation, tests, and configuration.
2. Preserve existing user work and follow existing project patterns where they exist.
3. If files already exist, modify them carefully instead of replacing unrelated work.

Files to create or modify:

src/abtp/strategies/base.py; src/abtp/strategies/engine.py; src/abtp/strategies/rules.py; docs/strategy_framework.md; tests/strategies/.

Architecture constraints:

- Preserve modular architecture, loose coupling, high cohesion, testability, observability, and environment-driven configuration.
- Do not place exchange API calls outside exchange adapter modules.
- Do not allow any order path to bypass the Risk Management Engine.
- Prefer spot trading first; leverage, margin, futures, and options stay disabled unless a later explicit stage enables them.
- Every decision must be explainable through stored inputs, generated signals, risk checks, and audit events.
- Use deterministic tests with fixtures; do not require real exchange credentials for unit tests.

Data and Bitcoin-dependency parameters:

Signal direction, entry reason, stop suggestion, target suggestion, timeframe, confidence,

regime, feature snapshot reference.

Minimum-risk controls:

Strategy output is Signal only. It cannot call exchange adapters or create orders directly.

Implementation requirements:

1. Keep the module narrowly scoped to this stage.
2. Define clear interfaces and data contracts.
3. Add documentation explaining assumptions, configuration, and operating limits.
4. Ensure future stages can extend the module without tight coupling.

Tests to add or run:

Strategy contract tests, deterministic fixture tests, disabled-strategy tests, signal audit tests.

Acceptance criteria:

Multiple strategies can be registered, enabled/disabled, and evaluated consistently.

Completion response:

Summarize created/modified files, tests run, remaining risks, and any assumptions. Do not claim the bot is profitable; report only verified behavior.

Stage 021 - Minimum-Risk Spot Strategy V1

Phase: Trading

You are Codex working in the ABTP repository. Implement Stage 021: Minimum-Risk Spot Strategy V1.

Objective:

Implement the first conservative BTC spot strategy focused on capital preservation and clear exits.

Prerequisites:

Stage 020 complete.

First actions:

1. Inspect the repository structure, existing documentation, tests, and configuration.
2. Preserve existing user work and follow existing project patterns where they exist.
3. If files already exist, modify them carefully instead of replacing unrelated work.

Files to create or modify:

src/abtp/strategies/min_risk_spot_v1.py; docs/strategies/min_risk_spot_v1.md;
tests/strategies/test_min_risk_spot_v1.py.

Architecture constraints:

- Preserve modular architecture, loose coupling, high cohesion, testability, observability, and environment-driven configuration.
- Do not place exchange API calls outside exchange adapter modules.
- Do not allow any order path to bypass the Risk Management Engine.
- Prefer spot trading first; leverage, margin, futures, and options stay disabled unless a later explicit stage enables them.
- Every decision must be explainable through stored inputs, generated signals, risk checks, and audit events.
- Use deterministic tests with fixtures; do not require real exchange credentials for unit tests.

Data and Bitcoin-dependency parameters:

Trend confirmation, RSI/momentum filter, ATR stop distance, volume confirmation, spread ceiling, regime filter, AI confidence as optional confirmation.

Minimum-risk controls:

Long-only spot, no leverage, no averaging down, required stop-loss, minimum reward-to-risk threshold, trade cooldown after losses.

Implementation requirements:

1. Keep the module narrowly scoped to this stage.
2. Define clear interfaces and data contracts.
3. Add documentation explaining assumptions, configuration, and operating limits.
4. Ensure future stages can extend the module without tight coupling.

Tests to add or run:

No-trade in bad regimes, signal in valid setup, stop/target generation, cooldown behavior.

Acceptance criteria:

Strategy produces explainable signals and rejects unclear conditions more often than it trades.

Completion response:

Summarize created/modified files, tests run, remaining risks, and any assumptions. Do not claim the bot is profitable; report only verified behavior.

Stage 022 - Risk Management Engine

Phase: Risk

You are Codex working in the ABTP repository. Implement Stage 022: Risk Management Engine.

Objective:

Implement the mandatory risk gate that approves, modifies, or rejects every signal before order creation.

Prerequisites:

Stage 021 complete.

First actions:

1. Inspect the repository structure, existing documentation, tests, and configuration.
2. Preserve existing user work and follow existing project patterns where they exist.
3. If files already exist, modify them carefully instead of replacing unrelated work.

Files to create or modify:

src/abtp/risk/engine.py; src/abtp/risk/rules.py; src/abtp/risk/position_sizing.py;
docs/risk_engine.md; tests/risk/.

Architecture constraints:

- Preserve modular architecture, loose coupling, high cohesion, testability, observability, and environment-driven configuration.
- Do not place exchange API calls outside exchange adapter modules.
- Do not allow any order path to bypass the Risk Management Engine.
- Prefer spot trading first; leverage, margin, futures, and options stay disabled unless a later explicit stage enables them.
- Every decision must be explainable through stored inputs, generated signals, risk checks, and audit events.
- Use deterministic tests with fixtures; do not require real exchange credentials for unit tests.

Data and Bitcoin-dependency parameters:

Capital, max risk per trade, stop distance, spread, slippage, fees, drawdown, daily/weekly P/L, open exposure, correlated positions, data quality.

Minimum-risk controls:

Default reject on missing stop, stale data, drawdown breach, daily loss breach, excessive spread/slippage, low confidence, or kill switch.

Implementation requirements:

1. Keep the module narrowly scoped to this stage.
2. Define clear interfaces and data contracts.
3. Add documentation explaining assumptions, configuration, and operating limits.
4. Ensure future stages can extend the module without tight coupling.

Tests to add or run:

Reject/approve fixtures, position-size formula tests, circuit-breaker tests, audit-reason tests.

Acceptance criteria:

No signal can become an OrderIntent without a RiskDecision containing explicit allow/reject reasons.

Completion response:

Summarize created/modified files, tests run, remaining risks, and any assumptions. Do not claim the bot is profitable; report only verified behavior.

Stage 023 - Portfolio and Position Manager

Phase: Risk

You are Codex working in the ABTP repository. Implement Stage 023: Portfolio and Position Manager.

Objective:

Track balances, positions, exposure, P/L, drawdown, cash reserve, and portfolio constraints.

Prerequisites:

Stage 022 complete.

First actions:

1. Inspect the repository structure, existing documentation, tests, and configuration.
2. Preserve existing user work and follow existing project patterns where they exist.
3. If files already exist, modify them carefully instead of replacing unrelated work.

Files to create or modify:

src/abtp/portfolio/manager.py; src/abtp/portfolio/accounting.py;
src/abtp/portfolio/exposure.py; docs/portfolio_manager.md; tests/portfolio/.

Architecture constraints:

- Preserve modular architecture, loose coupling, high cohesion, testability, observability, and environment-driven configuration.
- Do not place exchange API calls outside exchange adapter modules.
- Do not allow any order path to bypass the Risk Management Engine.
- Prefer spot trading first; leverage, margin, futures, and options stay disabled unless a later explicit stage enables them.
- Every decision must be explainable through stored inputs, generated signals, risk checks, and audit events.
- Use deterministic tests with fixtures; do not require real exchange credentials for unit

tests.

Data and Bitcoin-dependency parameters:

Balances, realized/unrealized P/L, exposure by asset, cash allocation, open orders, correlated exposure, drawdown.

Minimum-risk controls:

Maintain minimum cash reserve; block new positions when exposure or drawdown limits are breached.

Implementation requirements:

1. Keep the module narrowly scoped to this stage.
2. Define clear interfaces and data contracts.
3. Add documentation explaining assumptions, configuration, and operating limits.
4. Ensure future stages can extend the module without tight coupling.

Tests to add or run:

P/L calculations, drawdown tests, exposure aggregation, open-order reservation tests.

Acceptance criteria:

Risk engine can query a consistent PortfolioSnapshot before every decision.

Completion response:

Summarize created/modified files, tests run, remaining risks, and any assumptions. Do not claim the bot is profitable; report only verified behavior.

Stage 024 - Order Execution Engine in Paper-Safe Mode

Phase: Trading

You are Codex working in the ABTP repository. Implement Stage 024: Order Execution Engine in Paper-Safe Mode.

Objective:

Create an execution engine that converts risk-approved OrderIntent objects into simulated or adapter-routed orders.

Prerequisites:

Stage 023 complete.

First actions:

1. Inspect the repository structure, existing documentation, tests, and configuration.
2. Preserve existing user work and follow existing project patterns where they exist.
3. If files already exist, modify them carefully instead of replacing unrelated work.

Files to create or modify:

src/abtp/execution/engine.py; src/abtp/execution/order_router.py; src/abtp/execution/fills.py; docs/order_execution.md; tests/execution/.

Architecture constraints:

- Preserve modular architecture, loose coupling, high cohesion, testability, observability, and environment-driven configuration.
- Do not place exchange API calls outside exchange adapter modules.
- Do not allow any order path to bypass the Risk Management Engine.
- Prefer spot trading first; leverage, margin, futures, and options stay disabled unless a later explicit stage enables them.
- Every decision must be explainable through stored inputs, generated signals, risk checks, and audit events.

- Use deterministic tests with fixtures; do not require real exchange credentials for unit tests.

Data and Bitcoin-dependency parameters:

Order type, limit/market choice, estimated slippage, fees, partial fills, retries, idempotency key, exchange status.

Minimum-risk controls:

Default route is sandbox/paper. Live route must be impossible until the live gateway stage.

Implementation requirements:

1. Keep the module narrowly scoped to this stage.
2. Define clear interfaces and data contracts.
3. Add documentation explaining assumptions, configuration, and operating limits.
4. Ensure future stages can extend the module without tight coupling.

Tests to add or run:

Idempotent order submission, rejected order handling, partial fills, adapter failure, audit trail.

Acceptance criteria:

Only risk-approved orders are accepted and every order lifecycle event is recorded.

Completion response:

Summarize created/modified files, tests run, remaining risks, and any assumptions. Do not claim the bot is profitable; report only verified behavior.

Stage 025 - Backtesting Engine

Phase: Validation

You are Codex working in the ABTP repository. Implement Stage 025: Backtesting Engine.

Objective:

Build historical simulation for strategies using realistic fees, slippage, position sizing, and risk checks.

Prerequisites:

Stage 024 complete.

First actions:

1. Inspect the repository structure, existing documentation, tests, and configuration.
2. Preserve existing user work and follow existing project patterns where they exist.
3. If files already exist, modify them carefully instead of replacing unrelated work.

Files to create or modify:

src/abtp/backtesting/engine.py; src/abtp/backtesting/broker.py;
src/abtp/backtesting/slippage.py; docs/backtesting.md; tests/backtesting/.

Architecture constraints:

- Preserve modular architecture, loose coupling, high cohesion, testability, observability, and environment-driven configuration.
- Do not place exchange API calls outside exchange adapter modules.
- Do not allow any order path to bypass the Risk Management Engine.
- Prefer spot trading first; leverage, margin, futures, and options stay disabled unless a later explicit stage enables them.
- Every decision must be explainable through stored inputs, generated signals, risk checks, and audit events.

- Use deterministic tests with fixtures; do not require real exchange credentials for unit tests.

Data and Bitcoin-dependency parameters:

Historical candles, fees, spread/slippage model, risk rules, strategy signals, portfolio state, market regimes.

Minimum-risk controls:

Backtests must include costs and cannot report gross-only performance as success.

Implementation requirements:

1. Keep the module narrowly scoped to this stage.
2. Define clear interfaces and data contracts.
3. Add documentation explaining assumptions, configuration, and operating limits.
4. Ensure future stages can extend the module without tight coupling.

Tests to add or run:

Known scenario tests, no-lookahead tests, fee/slippage tests, drawdown halt tests.

Acceptance criteria:

A strategy can be replayed across historical BTC data with auditable trades and risk decisions.

Completion response:

Summarize created/modified files, tests run, remaining risks, and any assumptions. Do not claim the bot is profitable; report only verified behavior.

Stage 026 - Backtesting Reports and Performance Metrics

Phase: Validation

You are Codex working in the ABTP repository. Implement Stage 026: Backtesting Reports and Performance Metrics.

Objective:

Generate reports that judge strategies by risk-adjusted quality, not only profit.

Prerequisites:

Stage 025 complete.

First actions:

1. Inspect the repository structure, existing documentation, tests, and configuration.
2. Preserve existing user work and follow existing project patterns where they exist.
3. If files already exist, modify them carefully instead of replacing unrelated work.

Files to create or modify:

src/abtp/backtesting/reports.py; src/abtp/backtesting/metrics.py; docs/performance_metrics.md; tests/backtesting_reports/.

Architecture constraints:

- Preserve modular architecture, loose coupling, high cohesion, testability, observability, and environment-driven configuration.
- Do not place exchange API calls outside exchange adapter modules.
- Do not allow any order path to bypass the Risk Management Engine.
- Prefer spot trading first; leverage, margin, futures, and options stay disabled unless a later explicit stage enables them.
- Every decision must be explainable through stored inputs, generated signals, risk checks, and audit events.

- Use deterministic tests with fixtures; do not require real exchange credentials for unit tests.

Data and Bitcoin-dependency parameters:

Net return, max drawdown, profit factor, Sharpe/Sortino, win rate, expectancy, average win/loss, exposure time, regime performance.

Minimum-risk controls:

A strategy fails acceptance if drawdown or tail-loss rules fail, even if net return is positive.

Implementation requirements:

1. Keep the module narrowly scoped to this stage.
2. Define clear interfaces and data contracts.
3. Add documentation explaining assumptions, configuration, and operating limits.
4. Ensure future stages can extend the module without tight coupling.

Tests to add or run:

Metric formula tests, report snapshot tests, acceptance gate tests.

Acceptance criteria:

Reports clearly show whether a strategy is eligible for paper trading and why.

Completion response:

Summarize created/modified files, tests run, remaining risks, and any assumptions. Do not claim the bot is profitable; report only verified behavior.

Stage 027 - Paper Trading Engine

Phase: Validation

You are Codex working in the ABTP repository. Implement Stage 027: Paper Trading Engine.

Objective:

Run strategies against live market data with simulated execution, realistic fees, and full auditability.

Prerequisites:

Stage 026 complete.

First actions:

1. Inspect the repository structure, existing documentation, tests, and configuration.
2. Preserve existing user work and follow existing project patterns where they exist.
3. If files already exist, modify them carefully instead of replacing unrelated work.

Files to create or modify:

src/abtp/paper/engine.py; src/abtp/paper/account.py; src/abtp/paper/simulator.py;
docs/paper_trading.md; tests/paper/.

Architecture constraints:

- Preserve modular architecture, loose coupling, high cohesion, testability, observability, and environment-driven configuration.
- Do not place exchange API calls outside exchange adapter modules.
- Do not allow any order path to bypass the Risk Management Engine.
- Prefer spot trading first; leverage, margin, futures, and options stay disabled unless a later explicit stage enables them.
- Every decision must be explainable through stored inputs, generated signals, risk checks, and audit events.

- Use deterministic tests with fixtures; do not require real exchange credentials for unit tests.

Data and Bitcoin-dependency parameters:

Live candles/order book, simulated balances, estimated fills, slippage, fees, latency, risk limits, trade outcomes.

Minimum-risk controls:

Paper engine must use the same strategy, risk, portfolio, and execution interfaces intended for live trading.

Implementation requirements:

1. Keep the module narrowly scoped to this stage.
2. Define clear interfaces and data contracts.
3. Add documentation explaining assumptions, configuration, and operating limits.
4. Ensure future stages can extend the module without tight coupling.

Tests to add or run:

Simulated fill tests, live-data degradation tests, risk-halt tests, parity tests with execution contracts.

Acceptance criteria:

Paper trading can run unattended without real orders and produces live-like performance records.

Completion response:

Summarize created/modified files, tests run, remaining risks, and any assumptions. Do not claim the bot is profitable; report only verified behavior.

Stage 028 - Paper Trading Dashboard and API

Phase: Validation

You are Codex working in the ABTP repository. Implement Stage 028: Paper Trading Dashboard and API.

Objective:

Expose paper-trading state, signals, risk decisions, trades, P/L, and parameter health through API and dashboard.

Prerequisites:

Stage 027 complete.

First actions:

1. Inspect the repository structure, existing documentation, tests, and configuration.
2. Preserve existing user work and follow existing project patterns where they exist.
3. If files already exist, modify them carefully instead of replacing unrelated work.

Files to create or modify:

src/abtp/api/*.py; src/abtp/dashboard/; docs/api_design.md; docs/dashboard.md; tests/api/; tests/dashboard/.

Architecture constraints:

- Preserve modular architecture, loose coupling, high cohesion, testability, observability, and environment-driven configuration.
- Do not place exchange API calls outside exchange adapter modules.
- Do not allow any order path to bypass the Risk Management Engine.
- Prefer spot trading first; leverage, margin, futures, and options stay disabled unless a

later explicit stage enables them.

- Every decision must be explainable through stored inputs, generated signals, risk checks, and audit events.
- Use deterministic tests with fixtures; do not require real exchange credentials for unit tests.

Data and Bitcoin-dependency parameters:

Current BTC price, active regime, latest signals, blocked-trade reasons, data health, portfolio snapshot, drawdown.

Minimum-risk controls:

Dashboard is read-only for paper state except safe controls such as pause/resume and kill switch.

Implementation requirements:

1. Keep the module narrowly scoped to this stage.
2. Define clear interfaces and data contracts.
3. Add documentation explaining assumptions, configuration, and operating limits.
4. Ensure future stages can extend the module without tight coupling.

Tests to add or run:

API contract tests, dashboard render tests, kill-switch endpoint tests, authorization tests.

Acceptance criteria:

User can inspect why the bot did or did not trade during paper mode.

Completion response:

Summarize created/modified files, tests run, remaining risks, and any assumptions. Do not claim the bot is profitable; report only verified behavior.

Stage 029 - Logging, Monitoring, and Audit Trail

Phase: Production

You are Codex working in the ABTP repository. Implement Stage 029: Logging, Monitoring, and Audit Trail.

Objective:

Create structured logs, metrics, traces, and immutable audit records for all platform decisions.

Prerequisites:

Stage 028 complete.

First actions:

1. Inspect the repository structure, existing documentation, tests, and configuration.
2. Preserve existing user work and follow existing project patterns where they exist.
3. If files already exist, modify them carefully instead of replacing unrelated work.

Files to create or modify:

src/abtp/observability/logging.py; src/abtp/observability/metrics.py;
src/abtp/audit/events.py; docs/monitoring.md; tests/observability/.

Architecture constraints:

- Preserve modular architecture, loose coupling, high cohesion, testability, observability, and environment-driven configuration.
- Do not place exchange API calls outside exchange adapter modules.
- Do not allow any order path to bypass the Risk Management Engine.

- Prefer spot trading first; leverage, margin, futures, and options stay disabled unless a later explicit stage enables them.
- Every decision must be explainable through stored inputs, generated signals, risk checks, and audit events.
- Use deterministic tests with fixtures; do not require real exchange credentials for unit tests.

Data and Bitcoin-dependency parameters:

Data latency, signal count, risk rejects, order status, error rate, P/L, drawdown, exchange health, model version.

Minimum-risk controls:

Audit must capture risk rejection reasons and never hide failed or blocked trade attempts.

Implementation requirements:

1. Keep the module narrowly scoped to this stage.
2. Define clear interfaces and data contracts.
3. Add documentation explaining assumptions, configuration, and operating limits.
4. Ensure future stages can extend the module without tight coupling.

Tests to add or run:

Structured log tests, metric emission tests, audit persistence tests, sensitive-data redaction tests.

Acceptance criteria:

An operator can reconstruct every decision cycle from logs and audit tables.

Completion response:

Summarize created/modified files, tests run, remaining risks, and any assumptions. Do not claim the bot is profitable; report only verified behavior.

Stage 030 - Alerts and Notification Service

Phase: Production

You are Codex working in the ABTP repository. Implement Stage 030: Alerts and Notification Service.

Objective:

Notify the user about important states, blocked trades, losses, outages, and manual approval requests.

Prerequisites:

Stage 029 complete.

First actions:

1. Inspect the repository structure, existing documentation, tests, and configuration.
2. Preserve existing user work and follow existing project patterns where they exist.
3. If files already exist, modify them carefully instead of replacing unrelated work.

Files to create or modify:

src/abtp/notifications/service.py; src/abtp/notifications/channels.py; docs/notifications.md; tests/notifications/.

Architecture constraints:

- Preserve modular architecture, loose coupling, high cohesion, testability, observability, and environment-driven configuration.
- Do not place exchange API calls outside exchange adapter modules.

- Do not allow any order path to bypass the Risk Management Engine.
- Prefer spot trading first; leverage, margin, futures, and options stay disabled unless a later explicit stage enables them.
- Every decision must be explainable through stored inputs, generated signals, risk checks, and audit events.
- Use deterministic tests with fixtures; do not require real exchange credentials for unit tests.

Data and Bitcoin-dependency parameters:

Risk halt, kill switch, exchange outage, stale data, drawdown, large slippage, repeated losses, manual approval request.

Minimum-risk controls:

Critical alerts must be sent for live-mode risk events; alert failures must not permit trading.

Implementation requirements:

1. Keep the module narrowly scoped to this stage.
2. Define clear interfaces and data contracts.
3. Add documentation explaining assumptions, configuration, and operating limits.
4. Ensure future stages can extend the module without tight coupling.

Tests to add or run:

Channel adapter tests, alert throttling tests, critical-failure tests.

Acceptance criteria:

The system clearly notifies the user when risk controls pause or block the bot.

Completion response:

Summarize created/modified files, tests run, remaining risks, and any assumptions. Do not claim the bot is profitable; report only verified behavior.

Stage 031 - Security Hardening and Credential Protection

Phase: Production

You are Codex working in the ABTP repository. Implement Stage 031: Security Hardening and Credential Protection.

Objective:

Harden secrets, API permissions, authentication, authorization, dependency handling, and operational security.

Prerequisites:

Stage 030 complete.

First actions:

1. Inspect the repository structure, existing documentation, tests, and configuration.
2. Preserve existing user work and follow existing project patterns where they exist.
3. If files already exist, modify them carefully instead of replacing unrelated work.

Files to create or modify:

src/abtp/security/*.py; docs/security.md; docs/operations_security.md; tests/security/.

Architecture constraints:

- Preserve modular architecture, loose coupling, high cohesion, testability, observability, and environment-driven configuration.
- Do not place exchange API calls outside exchange adapter modules.

- Do not allow any order path to bypass the Risk Management Engine.
- Prefer spot trading first; leverage, margin, futures, and options stay disabled unless a later explicit stage enables them.
- Every decision must be explainable through stored inputs, generated signals, risk checks, and audit events.
- Use deterministic tests with fixtures; do not require real exchange credentials for unit tests.

Data and Bitcoin-dependency parameters:

Exchange API scopes, IP restrictions, encrypted secrets, user roles, audit access, dependency vulnerabilities.

Minimum-risk controls:

Use least-privilege exchange keys; withdrawal permissions must be unsupported and rejected by validation.

Implementation requirements:

1. Keep the module narrowly scoped to this stage.
2. Define clear interfaces and data contracts.
3. Add documentation explaining assumptions, configuration, and operating limits.
4. Ensure future stages can extend the module without tight coupling.

Tests to add or run:

Secret redaction tests, permission tests, unsafe-key detection, authz tests, dependency audit notes.

Acceptance criteria:

The system can run with trading-only keys and no withdrawal capability.

Completion response:

Summarize created/modified files, tests run, remaining risks, and any assumptions. Do not claim the bot is profitable; report only verified behavior.

Stage 032 - Supervised Live Trading Gateway

Phase: Production

You are Codex working in the ABTP repository. Implement Stage 032: Supervised Live Trading Gateway.

Objective:

Add a tightly controlled live gateway that requires manual approval and tiny position limits.

Prerequisites:

Stages 027-031 complete plus documented paper-trading success.

First actions:

1. Inspect the repository structure, existing documentation, tests, and configuration.
2. Preserve existing user work and follow existing project patterns where they exist.
3. If files already exist, modify them carefully instead of replacing unrelated work.

Files to create or modify:

src/abtp/live/gateway.py; src/abtp/live/approval.py; src/abtp/live/preflight.py;
docs/live_trading.md; tests/live/.

Architecture constraints:

- Preserve modular architecture, loose coupling, high cohesion, testability, observability, and environment-driven configuration.

- Do not place exchange API calls outside exchange adapter modules.
- Do not allow any order path to bypass the Risk Management Engine.
- Prefer spot trading first; leverage, margin, futures, and options stay disabled unless a later explicit stage enables them.
- Every decision must be explainable through stored inputs, generated signals, risk checks, and audit events.
- Use deterministic tests with fixtures; do not require real exchange credentials for unit tests.

Data and Bitcoin-dependency parameters:

Exchange balance, live fees, live spread, live slippage, approval token, order preview, current risk limits.

Minimum-risk controls:

Manual approval required; max risk per trade defaults to 0.25 percent; max one open live position until explicitly changed.

Implementation requirements:

1. Keep the module narrowly scoped to this stage.
2. Define clear interfaces and data contracts.
3. Add documentation explaining assumptions, configuration, and operating limits.
4. Ensure future stages can extend the module without tight coupling.

Tests to add or run:

Approval required tests, preflight rejection tests, live-mode lock tests with fake adapter.

Acceptance criteria:

No live order can be sent without explicit approval, passing preflight, and passing risk engine.

Completion response:

Summarize created/modified files, tests run, remaining risks, and any assumptions. Do not claim the bot is profitable; report only verified behavior.

Stage 033 - Limited Automation and Circuit Breakers

Phase: Production

You are Codex working in the ABTP repository. Implement Stage 033: Limited Automation and Circuit Breakers.

Objective:

Enable restricted automation only after evidence gates are met, with circuit breakers and immediate shutdown controls.

Prerequisites:

Stage 032 complete and manually reviewed small-live results.

First actions:

1. Inspect the repository structure, existing documentation, tests, and configuration.
2. Preserve existing user work and follow existing project patterns where they exist.
3. If files already exist, modify them carefully instead of replacing unrelated work.

Files to create or modify:

src/abtp/automation/controller.py; src/abtp/automation/circuit_breakers.py;
docs/automation_policy.md; tests/automation/.

Architecture constraints:

- Preserve modular architecture, loose coupling, high cohesion, testability, observability, and environment-driven configuration.
- Do not place exchange API calls outside exchange adapter modules.
- Do not allow any order path to bypass the Risk Management Engine.
- Prefer spot trading first; leverage, margin, futures, and options stay disabled unless a later explicit stage enables them.
- Every decision must be explainable through stored inputs, generated signals, risk checks, and audit events.
- Use deterministic tests with fixtures; do not require real exchange credentials for unit tests.

Data and Bitcoin-dependency parameters:

Paper-trading eligibility, live P/L, drawdown, consecutive losses, volatility shock, exchange health, operator presence.

Minimum-risk controls:

Automation must stop on daily loss, weekly loss, max drawdown, stale data, exchange outage, abnormal spread, or model/risk error.

Implementation requirements:

1. Keep the module narrowly scoped to this stage.
2. Define clear interfaces and data contracts.
3. Add documentation explaining assumptions, configuration, and operating limits.
4. Ensure future stages can extend the module without tight coupling.

Tests to add or run:

Circuit-breaker tests, eligibility-gate tests, pause/resume tests, kill-switch tests.

Acceptance criteria:

Automation cannot be enabled until evidence gates pass and can be stopped instantly.

Completion response:

Summarize created/modified files, tests run, remaining risks, and any assumptions. Do not claim the bot is profitable; report only verified behavior.

Stage 034 - Final Integration, Release Checklist, and User Guide

Phase: Production

You are Codex working in the ABTP repository. Implement Stage 034: Final Integration, Release Checklist, and User Guide.

Objective:

Perform end-to-end integration, documentation, deployment checklist, operator guide, and production readiness review.

Prerequisites:

Stages 005-033 complete.

First actions:

1. Inspect the repository structure, existing documentation, tests, and configuration.
2. Preserve existing user work and follow existing project patterns where they exist.
3. If files already exist, modify them carefully instead of replacing unrelated work.

Files to create or modify:

docs/final_integration.md; docs/user_guide.md; docs/deployment.md; docs/incident_response.md; docs/production_release_checklist.md; tests/e2e/.

Architecture constraints:

- Preserve modular architecture, loose coupling, high cohesion, testability, observability, and environment-driven configuration.
- Do not place exchange API calls outside exchange adapter modules.
- Do not allow any order path to bypass the Risk Management Engine.
- Prefer spot trading first; leverage, margin, futures, and options stay disabled unless a later explicit stage enables them.
- Every decision must be explainable through stored inputs, generated signals, risk checks, and audit events.
- Use deterministic tests with fixtures; do not require real exchange credentials for unit tests.

Data and Bitcoin-dependency parameters:

Complete lifecycle: data, features, prediction, strategy, risk, portfolio, execution, monitoring, alerts, dashboard.

Minimum-risk controls:

Production release requires all minimum-risk gates, kill switch, backup plan, incident response, and user acknowledgement.

Implementation requirements:

1. Keep the module narrowly scoped to this stage.
2. Define clear interfaces and data contracts.
3. Add documentation explaining assumptions, configuration, and operating limits.
4. Ensure future stages can extend the module without tight coupling.

Tests to add or run:

End-to-end paper flow, risk rejection flow, dashboard/API flow, security smoke, deployment smoke, restore drill.

Acceptance criteria:

ABTP is documented, tested, auditable, and ready only for controlled, minimum-risk operation.

Completion response:

Summarize created/modified files, tests run, remaining risks, and any assumptions. Do not claim the bot is profitable; report only verified behavior.

Final Readiness Gate

ABTP should be considered ready for controlled operation only when all stage acceptance criteria pass and the operator can inspect data health, signal reasoning, risk decisions, open exposure, account drawdown, alerts, and the kill switch from the dashboard.

- Backtests include costs, slippage, drawdown, and market-regime analysis.
- Paper trading demonstrates stable behavior for the agreed evaluation window.
- Live trading starts supervised, tiny, spot-only, and manually approved.
- Limited automation remains bounded by hard circuit breakers.
- The project documentation remains the single source of truth for future Codex work.